

MME8215: Operations Research - Seminar

IV. Mixed Integer Programming.

Contents

1	Modelling power	2
1.1	Integer modelling tricks	2
2	Examples	4
2.1	Incredible Chairs	4
2.2	Startup Fund	5
2.3	Student Teacher Meetings	8
3	Problem set	11

LP models are very useful, but there are many situations where the planning problem in question cannot be modelled sufficiently precisely. In many problems fractionality of the variables is problematic. In a Mixed Integer Programming (MIP) model, the variables are restricted, only allowing integer values as described in the domain constraints.

As an example consider the Incredible Chairs problem again.

Example 1. The company Incredible Chairs produces two different types of chairs, A and B , where one unit (pallet of chairs) of Chair A can be sold for a profit of 4 and one unit of Chair B can be sold for a profit of 6. The chairs are produced on three production lines:

1. Line 1 requires 2 hours to produce one unit of Chair A , and it can be used at most 14 hours due to other production on the line.
2. Line 2 requires 3 hours to produce one unit of Chair B , and it can be used at most 15 hours.
3. Line 3 requires 4 hours to produce one unit of Chair A and 3 hours to produce one unit of chair B , and it can be used at most 36 hours.

How would you plan the production of the chairs?

The LP model is:

$$\begin{aligned}
 \min \quad & z = 4 \cdot x_A + 6 \cdot x_B \\
 \text{subject to} \quad & \\
 & 2 \cdot x_A \leq 14, \\
 & 3 \cdot x_B \leq 15, \\
 & 4 \cdot x_A + 3 \cdot x_B \leq 36, \\
 & x_A, x_B \geq 0.
 \end{aligned}$$

If we solve this problem we obtain: maximal profit 51.0 and the optimal values of the two decision variables, $x_A = 5.25$ and $x_B = 5.0$.

If in the above example we impose the requirement that $x_A, x_B \in \mathbb{Z}^+$, i.e. the chairs need to be produced in whole numbers, the model becomes a MIP model. The last line of the model becomes $x_A, x_B \in \mathbb{Z}^+$ instead of $x_A, x_B \geq 0$.

1 Modelling power

At first glance, the addition of integer requirements for the variables does not seem like such a big change. However, many problems are inherently integer:

- Chair production, or any type of production of integer amounts - It is hard to sell fractional chairs.
- Vehicle routing, e.g. trucks, ships, trains - a fractional ship cannot sail.
- Manpower planning - planning with fractional workers is not a good option.
- Timetabling - the classes could be divided up, but what about the teacher?

It turns out that there are many cases where one or more variables needs to be integer ($x \in \mathbb{Z}$) or binary ($x \in \{0, 1\}$).

In general, just rounding fractional variables is not guaranteed to reach an optimal solution, perhaps not even a feasible solution.

Generating a good integer solution from the Incredible chairs model is not complicated. The optimal value for Incredible chairs is 51 with the values $x_A = 5.25$ and $x_B = 5$. Because all the coefficients of x_A are positive in the constraints and all the constraints are less-equal $\leq$, x_A can simply be rounded down to $x_A = 5$, achieving an integer solution. This reduce the objective value to 50. But is this solution optimal?

In general, just rounding fractional variables is not guaranteed to reach an optimal solution, perhaps not even a feasible solution. In the Incredible Chairs case, the integer solution objective went from 51 to 50. In other cases, the changes in the objective can be far greater: Suppose a container ship has to be routed to pick up containers. If 3 containers have to be picked up from a port by a container ship with a capacity of 1000, an LP model will assign a $\frac{3}{1000}$ ship to do the job. If what is minimized is cost, the objective can increase substantially by increasing the value from $\frac{3}{1000}$ to 1.

An important special case of integer variables is binary variables i.e. $x \in \{0, 1\}$. This can model yes/no decisions, usually interpreting 1 as yes and 0 as no.

1.1 Integer modelling tricks

There are a number of standard modelling tricks which can be utilized when modelling with integer or binary decision variables. Here we briefly mention the most important constructions, and point to the assignments where they are needed.

Fixed charge link: Often a resource like a ship, depot, truck or plant, is represented by a binary variable (is it needed or not) or how many do we need. The cost of using the resource is then 0 if it is not used, but even the most limited use incurs a fixed cost. This can be stated as:

$$h(x) = \begin{cases} 0 & , x = 0, \\ f + px & , x > 0. \end{cases}$$

If we assume that there is an upperbound on $x \leq B$, then this can be modelled using binary decision variables in the following way:

$$\begin{aligned}h(x) &= fy + px, \\x &\leq By, \\y &\in \{0, 1\}.\end{aligned}$$

Such a fixed charge link could, e.g., correspond to the cost of transporting x containers on a container ship: There is an additional cost, due to extra fuel needed for the extra weight of the container, but the main cost is probably due to the need for a full ship, i.e. the y cost.

If the resource can be increased in fixed amounts, then the only change needed is to change the definition of the resource variable from binary to integer. This could correspond to, e.g., a number of container ships.

Logic on binary variables Often binary variables can be used to formulate logical relations between different binary variables. Usually, a binary variable $x = 1$ corresponds to true and $x = 0$ corresponds to false. With these definitions, we can easily define the various logic operators:

- **OR:** Given two binary variables x and y , define the v binary variable as x OR y :

$$\begin{aligned}v &\geq x, \\v &\geq y, \\v &\leq x + y.\end{aligned}$$

- **AND:** Given two binary variables x and y , define the v binary variable as x AND y :

$$\begin{aligned}v &\leq x, \\v &\leq y, \\v &\geq x + y - 1.\end{aligned}$$

- **NOT:** Given one binary variable x , define the v binary variable as NOT x :

$$v = 1 - x.$$

- **Force selection:** Given a set of i items, select at least 3 of these:

$$\sum_i x_i \geq 3.$$

- **Limit selection:** Given a set of i items, select at most 4 of these:

$$\sum_i x_i \leq 4.$$

A classical mistake in modelling is to try to model logic by using an API variables (e.g. JuMP) as variables. Perhaps this is not possible, when logic in the model needs to be modelled, it can be modelled as above.

Binary multiplication: If a MIP problem can be modelled using linear models, i.e. where the variables are not multiplied, given two binary variables x and y the multiplication of these can be formulated in a linear fashion, where the binary variable z is the result: (exactly as the x AND y operator):

$$\begin{aligned}z &\leq x, \\z &\leq y, \\z &\geq x + y - 1.\end{aligned}$$

2 Examples

2.1 Incredible Chairs

Reformulate the LP model from the Incredible Chairs problem so that it becomes a MIP model in which only discrete quantities of chairs are considered.

Solution: just change the type of decision variables in the model of the Incredible Chairs problem.

Below is a solution modelled in *JuMP* and solved with *HiGHS* in the *Julia* language.

```
using JuMP
using HiGHS

IC = Model(HiGHS.Optimizer)

@variable(IC, xA >= 0, Int)
@variable(IC, xB >= 0, Int)
@objective(IC, Max, 4*xA + 6*xB)
@constraint(IC, 2*xA <= 14)
@constraint(IC, 3*xB <= 15)
@constraint(IC, 4*xA + 3*xB <= 36)

print(IC)
optimize!(IC)

println("Termination status: $(termination_status(IC))")
if termination_status(IC) == MOI.OPTIMAL
    println("Optimal objective value: $(objective_value(IC))")
    println("xA: ", value(xA))
    println("xB: ", value(xB))
else
    println("No optimal solution available")
end
```

And the result:

```
Solving report
Status          Optimal
Primal bound    50
Dual bound      50
Gap             0% (tolerance: 0.01%)
P-D integral    0
Solution status feasible
                50 (objective)
                0 (bound viol.)
                0 (int. viol.)
                0 (row viol.)
Timing          0.00
Max sub-MIP depth 0
Nodes           0
Repair LPs      0
```

```

LP iterations      0
Termination status: OPTIMAL
Optimal objective value: 50.0
xA: 5.0
xB: 5.0

```

2.2 Startup Fund

A startup fund manager decides to use MIP models in order to optimize their profit. A new investment fund is just starting up, and €100 million is available for investment. The startup fund staff have identified nine possible investment opportunities in startup companies, the core information being the required investment and the expected value increase in 3 years.

	Investment								
	1	2	3	4	5	6	7	8	9
Estimated profit factor	1.7	1.4	1.3	2.1	1.9	1.8	1.5	2.2	1.6
Capital required	17	25	19	25	28	23	29	31	18

Investment details (millions of dollars).

Given an available budget of €100 million, the objective for the fund is to select the best possible set of investments. Since these investment opportunities are in startup companies, the full amount of capital needed for an investment must be invested (and it is not possible to invest more). Furthermore, money which is not invested is considered lost.

Formulate a binary integer program that can be used to solve this problem.

Solution: The investment manager has a classical decision: What to invest in. Since it is not possible to divide the investment in smaller sizes, making an investment is modelled with a binary variable. Then the expected profit should be maximized constrained by the budget. This problem is the classical OR problem, the knapsack problem.

Sets: $i \in Investments = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ - The set of different investment possibilities.

Parameters:

- $EstProfitFactor_i$: Investment profit factor for investment i
- $CapitalRequirements_i$: Capital requirement for project i
- $Budget = 100$: The available capital

Decision variables: Invest or not in project i : $x_i \in \{0, 1\}$. $x_i = 1$ models that an investment is made and $x_i = 0$ models that an investment is not made.

Model:

$$\begin{aligned}
 \max \quad & z = \sum_i EstProfitFactor_i \cdot x_i \\
 \text{subject to} \quad & \sum_i CapitalRequirements_i \leq Budget, \forall i.
 \end{aligned}$$

Below is a solution modelled in JuMP and solved with HiGHS in the Julia language

```

using JuMP
using HiGHS

# PARAMETERS
Investments = [1 2 3 4 5 6 7 8 9]
I=length(Investments)

# Data
EstProfitFactor = [1.7 1.4 1.3 2.1 1.9 1.8 1.5 2.2 1.6]
CapitalRequirements = [17 25 19 25 28 23 29 31 18]
Budget = 100

# Model
Investment = Model(HiGHS.Optimizer)
@variable(Investment, x[1:I], Bin) # 1 if investment is made in i

# Maximize est-profit
@objective(Investment, Max,
           sum(EstProfitFactor[i]*x[i] for i=1:I) )

# budget constraint
@constraint(Investment,
           sum(CapitalRequirements[i]*x[i] for i=1:I) <= Budget)

# solve
optimize!(Investment)

# Report results
if termination_status(Investment) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(Investment))")
    for i in 1:I
        println("Investment $(Investments[i]) $(round{Int8,value(x[i])})")
    end
else
    println(" No solution")
end

```

And the result:

```

Solving report
Status          Optimal
Primal bound    7.8
Dual bound      7.8
Gap             0% (tolerance: 0.01%)
P-D integral    0.00407696755567
Solution status feasible
               7.8 (objective)
               0 (bound viol.)
               0 (int. viol.)
               0 (row viol.)
Timing          0.01
Max sub-MIP depth 1

```

```

Nodes          1
Repair LPs     0
LP iterations   8
                0 (strong br.)
                7 (separation)
                0 (heuristics)

```

RESULTS:

```

objective = 7.8000000000000001
Investment 1 1
Investment 2 0
Investment 3 0
Investment 4 1
Investment 5 0
Investment 6 1
Investment 7 0
Investment 8 1
Investment 9 0

```

Additional assignment: After creating the first investment plan, the manager decides that investments in 1 and 4 are competing companies, so it is not possible to invest in both companies and the same applies for investments in 6 and 8. Furthermore, the investments in 6 and 9 can only be performed if either investment 2 or investment 3 is done, since the developed technology in these is the basis of 6 and 9.

Solution: The additional constraints can be modeled as

```

@constraint(Investment, x[1] + x[4] <= 1)
@constraint(Investment, x[6] + x[8] <= 1)
@constraint(Investment, x[6] + x[9] <= 2*(x[2] + x[3]))

```

And the result:

Solving report

```

Status          Optimal
Primal bound    7.3
Dual bound      7.3
Gap             0% (tolerance: 0.01%)
P-D integral    0.00238953303715
Solution status feasible
                7.3 (objective)
                0 (bound viol.)
                0 (int. viol.)
                0 (row viol.)

Timing          0.02
Max sub-MIP depth 1
Nodes           1
Repair LPs      0
LP iterations    14
                0 (strong br.)
                9 (separation)
                0 (heuristics)

```

RESULTS:

`objective` = 7.3000000000000001

`Investment` 1 0

`Investment` 2 1

`Investment` 3 0

`Investment` 4 1

`Investment` 5 0

`Investment` 6 0

`Investment` 7 0

`Investment` 8 1

`Investment` 9 1

2.3 Student Teacher Meetings

In Danish schools, it is customary to have meetings during the school year, often twice a year, between the student and parents and some of the teachers of the student. The meetings take place in the evening. Before the student-teacher meeting evening, the student (or parents!) make a list of the teachers they would like to meet with. When all students have submitted their teacher meeting request, a schedule needs to be made.

The teachers each use their own classroom for their meetings. The students and their parents walk around the school and meet with the teachers they wish to talk to. A number of time slots are given, e.g., a 10 minute interval, and your job is simply to plan when, i.e. in which time slot, the meetings will occur.

The meetings are private, so a teacher can only meet one family at a time. The families would like to spend as little time as necessary on the meetings. Therefore, the overall objective is to minimize the sum of the latest time slot indices assigned to the families.

The problem is surprisingly difficult, so the data sets are all rather small and open-source solvers all struggle on even the smallest instances. It is very likely that the solver will not find the optimal solution and will terminate with a solution that is not optimal, or even without a solution.

Formulate the student-teacher meeting planning problem as a MIP, minimizing the sum of late time slots.

Solution The student teacher meeting is a scheduling problem, i.e. a problem where events are scheduled to take place at a specific place at a specific time. This problem is actually rather hard to solve, and the problem really shows the difference between the open-source solvers and commercial solvers.

Sets:

- Students: $s \in Students = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\}$
- Teachers: $t \in Teachers = \{1, 2, 3, 4, 5, 6\}$
- Timeslots: $ts \in TimeSlots = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$

Parameters: $StudentTeacherMeetings_{s,t}$: Incidence matrix, where

$StudentTeacherMeetings_{s,t} = 1$ if student s has to have a meeting with teacher t , otherwise 0.

Decision variables:

- Should student s meet teacher t in timeslot ts : $x_{t,ts}^s \in \{0, 1\}$

- The latest meeting for student $s : y_s \geq 0$.

Model:

$$\begin{aligned} \min \quad & z = \sum_s y_s \\ \text{subject to} \quad & \sum_{ts} x_{t,ts}^s = \text{StudentTeacherMeetings}_{s,t}, \forall s, t, \\ & \sum_t x_{t,ts}^s = 1, \forall s, ts \text{ (each student can only meet one teacher at a time),} \\ & \sum_s x_{t,ts}^s = 1, \forall t, ts \text{ (each teacher can only meet one student at a time),} \\ & ts \cdot \sum_t x_{t,ts}^s \leq y_s, \forall s, ts \text{ (force the value of } y_s \text{ to take the value of the last timeslot } ts). \end{aligned}$$

Below is a solution modelled in *JuMP* and solved with *HiGHS* in the *Julia* language

```
using JuMP
using HiGHS
# Data
include("s_04_data_15_6.jl")

# S: no of students      : in data file
# T: no of teachers      : in data file
# TS: no of timeslots    : in data
total_no_meetings=sum(StudentTeacherMeetings[:,:])
println("Total no. of meetings: $(total_no_meetings)")

# Model
stmeet = Model(HiGHS.Optimizer)

# 1 if student s has a meeting with teacher t in time ts
@variable(stmeet, x[1:S,1:T,1:TS], Bin)

for s=1:S
    for t=1:T
        if StudentTeacherMeetings[s,t]==0
            for ts=1:TS
                fix(x[s,t,ts],0; force = true)
            end
        end
    end
end

# last meeting of student s
@variable(stmeet, y[1:S] >= 0)

# Minimize summed Inconvenience
@objective(stmeet, Min, sum( y[s] for s=1:S ) )

# meetings has to occur (or not occur)
```

```

@constraint(stmeet, [s=1:S,t=1:T], sum( x[s,t,ts] for ts=1:TS ) == StudentTeacherMeetings[s,t])

# only one meeting pr timeslot for each student s
@constraint(stmeet, [s=1:S,ts=1:TS], sum( x[s,t,ts] for t=1:T ) <= 1)

# only one meeting pr timeslot for each teacher t
@constraint(stmeet, [t=1:T,ts=1:TS], sum( x[s,t,ts] for s=1:S ) <= 1)

# force value of y to last meeting for each student
@constraint(stmeet, [s=1:S,ts=1:TS], ts*sum( x[s,t,ts] for t=1:T ) <= y[s])

# solve
optimize!(stmeet)
println("-----");
if termination_status(stmeet) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(stmeet))")
    println("Minimal num of meetings: $(sum(StudentTeacherMeetings[:,:]))")
    println("Solve time: $(solve_time(stmeet))")
else
    println(" No solution")
end

```

The data file contains the matrix and inputs.

```

StudentTeacherMeetings=[
 0 1 0 0 1 1;
 0 0 1 0 1 1;
 1 0 0 1 0 0;
 0 1 1 0 0 0;
 1 0 0 1 0 0;
 1 1 0 1 0 0;
 1 1 1 0 0 0;
 0 0 0 0 1 1;
 0 0 0 1 1 1;
 0 0 0 1 1 1;
 0 0 1 0 1 1;
 0 1 0 0 1 1;
 0 1 1 0 0 0;
 0 1 1 0 0 0;
 1 0 0 1 1 0
]
S=15
T=6
TS=11

```

And the solution:

```

Students: 15 Teachers: 6 Timeslots: 11
Total no. of meetings: 39
...
Solving report

```

```

Status                Optimal
Primal bound          66
Dual bound            66
Gap                   0% (tolerance: 0.01%)
P-D integral          2.98462585579
Solution status       feasible
                      66 (objective)
                      0 (bound viol.)
                      3.99680288865e-15 (int. viol.)
                      0 (row viol.)
Timing                28.06
Max sub-MIP depth     4
Nodes                 4615
Repair LPs            0
LP iterations          477562
                      142513 (strong br.)
                      35754 (separation)
                      27198 (heuristics)
Termination status:  OPTIMAL
-----
RESULTS:
objective = 66.0
Minimal num of meetings: 39
Solve time: 28.066952082997886
-----

```

3 Problem set

1. Work-plan for Teaching Assistants A Mathematical Programming crash course, over 9 days, is taught by 4 teaching assistants (TAs) and 2 professors. The professors give the lectures and the TAs are available for help with the exercises. Suppose that four TAs (AL, FR, JE, and MI) are the TAs in the course. In this assignment, you have to plan when they should work during the course. Each TA must work exactly 52 hours (a total of 208 hours are available for the four TAs). The professors have attempted to come up with a plan of how many TAs to assign for exercise help in each of the 8 time slots from 9 to 17. This TA demand is given below, [1](#).

To make the work easier for the TAs, we wish to take into account their personal preferences. Each TA has specified inconvenience scores for each day d and time period t in the matrix of inconvenience scores $Inconvenience_{t,d}^{ta}$ as given in the text file `s_04_inconvenience.txt`.

Formulate a mathematical model that can be used to find the optimal work plan for the TAs.

When you inspect the generated plan, you observe that the TAs work several times on each day. This is rather inconvenient. You decide to modify your model and include the restrictions that each TA can only work consecutive hours and that if a TA works on a particular day, then the TA has to work at least two hours.

Revise the mathematical model with additional restrictions.

2. Printing Precision Parts You work for a company that manufactures high quality polymer parts for OrgoCorp, a galactic bioengineering company. The quality requirements are so high that you need to use very sophisticated 3D printing machines. You need to develop a mathematical

Figure 1: TA demand for the first 9 days.

	Day								
	1	2	3	4	5	6	7	8	9
9-10	4	4	4	0	3	0	3	0	2
10-11	4	4	4	2	4	2	4	2	2
11-12	4	4	4	4	4	4	4	4	2
12-13	4	3	3	3	3	4	3	3	1
13-14	4	3	3	3	3	4	3	3	1
14-15	4	3	3	3	3	4	3	3	1
15-16	3	3	2	3	3	4	3	3	1
16-17	3	3	2	2	2	3	2	2	1

programming model to organize monthly production considering:

- *Fixed operational costs* (euros): Each 3D printing machine m has a different fixed operational costs Fm that needs to be considered when a printing machine is used to print any units.
- *Unitary production costs* (euros): Each 3D printing machine m produces parts of product type p at a different unitary costs C_{mp} .
- *Capacity* (minutes): Each 3D printing machine m has a different capacity S_m (minutes).
- *Speed* (minutes/part): Each 3D printing machine m prints a unit of product type p at a different unitary speed.
- *Demand* (units): OrgoCorp has confirmed a demand for d_p units of every product type p .

Build a model that takes into account these requirements, identifying indices, decision variables, objective function and constraints.

The CEO of OrgoCorp, The High Evolutionary, would like to set up a new contract allowing more flexibility in the delivery of units, such that:

- *Demand per period* (units): Now you need to consider that the demand for every product type p is not constant and depends on the planning period t , D_{pt} .
- *Delayed demand* (units): You may delay the production for one period. For every unit of product type p that is delayed, OrgoCorp will charge an extra delayed demand cost of B_{pt} .

Modify the model to take into account this new contract.

3. Fleet Assignment In the distant future, tensions between Earth, Mars, and the Belt are at an all-time high. The United Nations Navy (UNN) must deploy vessels to ensure the stability in earth's colony and provide security to supply chain operations across the solar system.

More specifically, Earth needs to assign five available warships to secure five colonies in need of military assistance: Ceres, Ganymede, Europa, Io, and Callisto.

Each vessel has an associated cost for each mission, that depends on factors such as suitability, operational range, and efficiency. The costs are provided in the table below:

Requirements:

Vessel / Colony	Ceres	Ganymede	Io	Europa	Callisto
UNN Agatha King	12	18	25	30	14
UNN Nathan Hale	20	14	22	24	18
UNN Donnager	30	12	18	16	10
UNN Rocinante	15	22	28	12	20
UNN Behemoth	18	20	30	14	12

1. Formulate an Integer Programming model to minimize the total cost of vessel assignments while ensuring every colony gets a vessel.
2. Draw a graph and solve the problem using graph algorithms (as a transportation model).
3. Describe a strategy to design a greedy algorithm to solve the problem.